

一. 问题分析

1. 分析并确定要处理的对象是什么

处理的对象是有向图。通过读入输入的顶点与边构造一个有向图进行处理

2. 分析并确定要实现的功能是什么

求解从节点1到每一个节点的连通性与路径数。

3. 分析并确定处理后的结果如何显示

每个用例的结果占一行，包含n个整数，对应各顶点状态

0: 从顶点 1 到 v 不可达

1: 从顶点 1 到 v 只有一条路径

2: 从顶点 1 到 v 有超过一条路径，且路径数是有限的

-1: 从顶点 1 到 v 有无穷多条路径

4. 用题目中样例，详细给出样例求解过程

5	
6	7
1	4
1	3
3	4
4	5
2	1
5	5
5	6

处理过程：

1. 可达性分析：从 1 出发，经边 $(1,3) \rightarrow (3,4) \rightarrow (4,5) \rightarrow (5,5) \rightarrow (5,6) \dots$ 可达 3,4,5,6；顶点2不可达，其余可达

2. SCC分解：

$\{1\}$, $\{3\}$, $\{4\}$, $\{5\}$, $\{6\}$ （每个顶点自成一个SCC）

3. 环检测：顶点 5 有自环 $(5,5)$ ，故 5 所在强连通分量（SCC）含环

4. 无限路径传播：由于 $5 \rightarrow 6$ ，环影响可传递至 6。

5. 有限路径计数：

$1 \rightarrow 1$: 1条

$1 \rightarrow 2$: 不可达 0条

$1 \rightarrow 3$: 1条

$1 \rightarrow 4$: 2条 ($1 \rightarrow 3 \rightarrow 4$) 与 ($1 \rightarrow 4$)

$4 \rightarrow 5$: 不计算（已标记为无限）

6. 最终输出：1 0 1 2 -1 -1

二. 数据结构与算法设计

1. 抽象数据类型

ADT Graph

数据对象：非空有限集合 V 与有限集合 $E \subseteq V \times V$ 。

基本操作：

AddEdge(u, v) —— 插入弧。

Adj(v) —— 返回从 v 出发的后继表。

ADT SCC Graph

数据对象：Graph 的强连通分量集合 $C = \{C_1 \cdots C_k\}$ ，以及分量间 DAG G_C 。

基本操作：

Tarjan() —— 线性时间求 SCC 并返回编号数组 sccId[]。

HasCycle(id) —— 判定分量自成环。

SuccScc(id) —— 获取分量 id 的后继分量列表。

2. 物理数据对象设计：

vector<vector<int>> adj($n + 1$) 邻接表，存储图的边关系

vector<bool> canReach($n + 1$) 可达标记，记录结点是否由1可达

stack<int> stk Tarjan算法栈，维护当前处理的顶点序列

vector<vector<int>> sccList SCC编号表，记录结点的SCC编号

vector<vector<int>> sccEdges(sccCnt)分量 DAG 邻接表

vector<int> inDegree($n + 1$), pathCount($n + 1$) 路径计数数组，存储有限路径数量

3. 算法思想的设计

本算法通过分层处理（分为无限路径与有限路径）策略解决有向图路径问题：首先用BFS标记从1可达的节点，接着通过Tarjan算法分解强连通分量，然后识别环结构，构建分量级联图后以BFS传播无限路径标记，最后在剔除有环结点的子图上拓扑排序动态维护路径计数，采用路径数截断策略避免溢出（路径数达到2后，后续便不再增加），最后输出每个用例的路径数目。

补充：Tarjan算法在本题中求SCC过程

● 初始化阶段

创建三个核心数组：

1. dfn[x]：记录节点x的访问时间戳（从1开始递增）

2. low[x]：存储x通过树边/回边能访问到的最早节点时间戳

3. inStack[x]：标记节点是否在当前处理栈中

准备两个栈：

stk：存储当前正在处理的节点序列

s（局部栈）：模拟递归过程的双状态处理栈

● 非递归DFS处理

```
stack<pair<int, bool>> s;
s.push({x, false});
首次访问阶段 (done=false) :
设置dfn[x] = low[x] = ++ts
将节点压入Tarjan栈stk, 标记inStack[x] = true
将邻居逆序压栈 (保证处理顺序与传统递归一致)
● 回溯处理阶段 (done=true) :
遍历所有邻接节点y:
若y在栈中 (inStack[y] == true)
low[x] = min(low[x], low[y]); 将当前节点的low值更新为可达的最小时间戳
● 识别SCC
当某节点的dfn与low相等时, 说明它是当前SCC的根节点
将栈中该节点之上的所有元素归入同一个SCC
● 更新父结点low值
if (!s.empty())
low[s.top().first] = min(low[s.top().first], low[x]);
在回溯过程中持续更新父节点的low值, 确保能正确传递最小可达时间戳
```

由此, 可高效寻找图中SCC。

4. 关键步骤的算法步骤

算法可分为三个阶段

4.1 可达性判断

使用BFS标记从1可达的结点

4.2 环与无穷路径判定

4.2.1 强连通分量分解:

使用Tarjan算法高效求解强连通分量, 得到顶点到分量的映射 $sccId[v]$, 对含环的SCC进行标记, 以及分量有向无环图 (SCC DAG)

4.2.2 确定含环分量

若某分量内部顶点数 ≥ 2 , 或者包含自环边 (v, v) , 则该分量内存在可无限复用的回路, 记为 含环分量。

4.2.3 传递闭包 把所有含环分量作为源点, 在图上再做一次 BFS。凡是能到达这些源点的分量同样能够把路径连接到回路形成无穷条路径, 因此把这部分顶点整体标记为-1。

完成上述步骤, 剩余未标记的顶点所构成的子图必定不存在任何有向环, 它们形成一张真正的 DAG。

4.3 处理有限路径

4.3.1 得到拓扑排序 topoQ

4.3.2 动态规划计数

令 $pathCount[nex]$ 表示从 1 到 nex 的路径条数, 初始化 $pathCount[1] = 1$ (空路径视为一条)。按 topo 顺序扫描顶点 nex , 将 $pathCount[nex]$ 累加到其后继 u :

`pathCount[nex] = min(2, pathCount[nex] + pathCount[cur]);`

一旦 `pathCount[nex]` 达到 2, 后续便不再增加, 因为题目只需要知道 "是否多于一条", 无需精确计数。

最后遍历输出即可。

三. 算法性能分析

1. 时间复杂度

BFS可达性分析, 每个顶点和边访问一次 时间复杂度为 $O(n + m)$

Tarjan算法, 深度优先扫描子图 时间复杂度为 $O(n + m)$

含环结点在SCC-DAG上传播 时间复杂度为 $O(n + m)$

拓扑排序计数, 入度队列迭代 时间复杂度为 $O(n + m)$

故总时间复杂度为 $O(n + m)$

2. 空间复杂度

邻接表 `adj`: 存储 m 条边, 大小 $O(n + m)$ 。

SCC- DAG: 至多与原图同阶, $O(n + m)$ 。

辅助数组: `dfn`, `low`, `sccId`, `canReach`, `state`, `cnt` 等, 各占 $O(n)$

总体来看, 空间复杂度为 $O(n + m)$